

Assignment 10.1 Ai Assisted Coding

Name: Shahalam

Htno:2303a510C1

Btno:06

Task 1: Syntax and Logic Errors Use AI to identify and fix syntax and logic errors in a faulty Python script.

Prompt:

Act as a Python debugging expert. Analyze the faulty Python script below and:

- Identify syntax errors and logic errors
- Fix the code so it runs correctly
- Explain the corrections briefly • Show the corrected runnable code Faulty code:

Calculate average score of a student def

```
calc_average(marks):
```

```
total = 0 for m
```

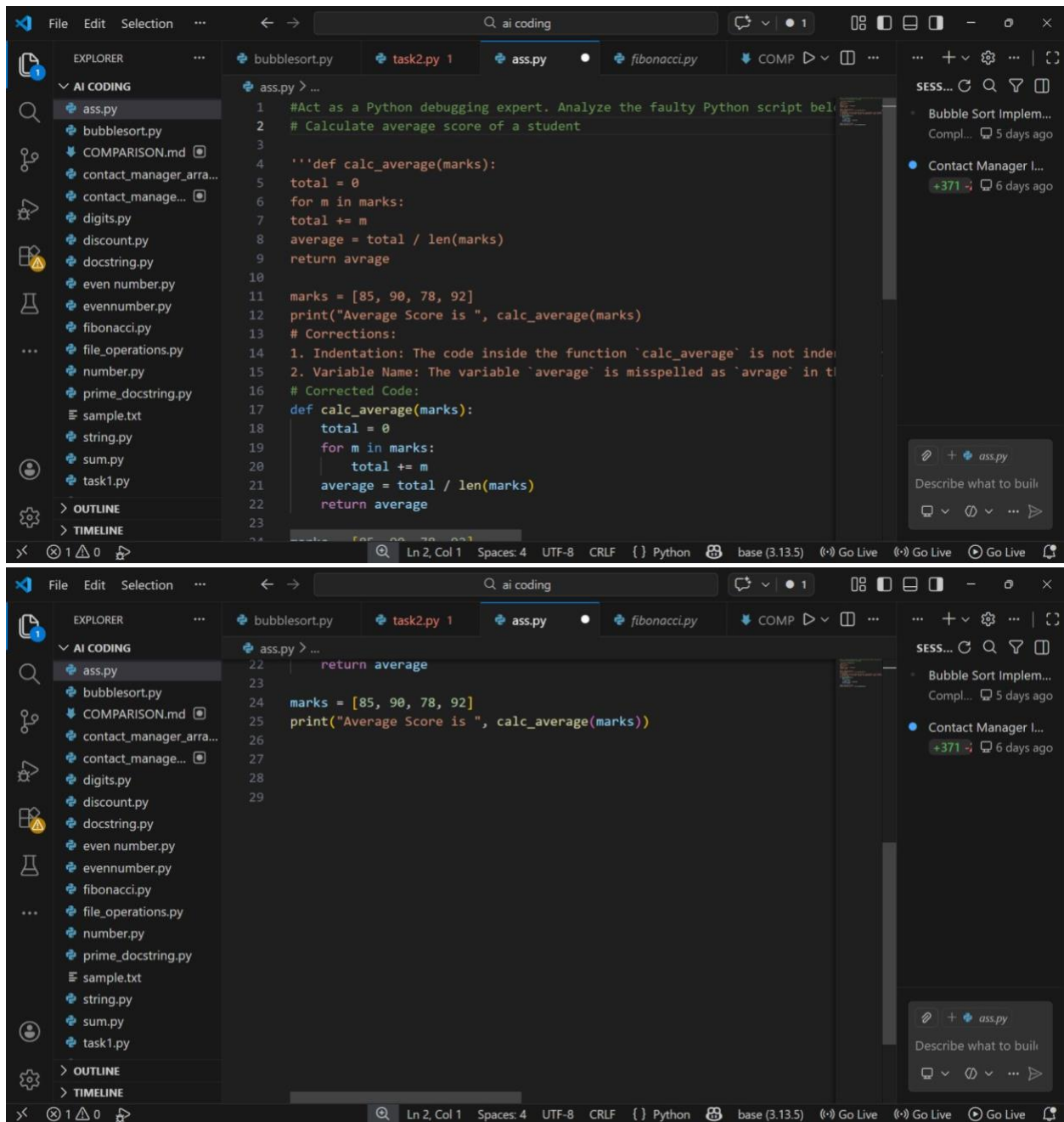
```
in marks:
```

```
total += m average = total / len(marks) return
```

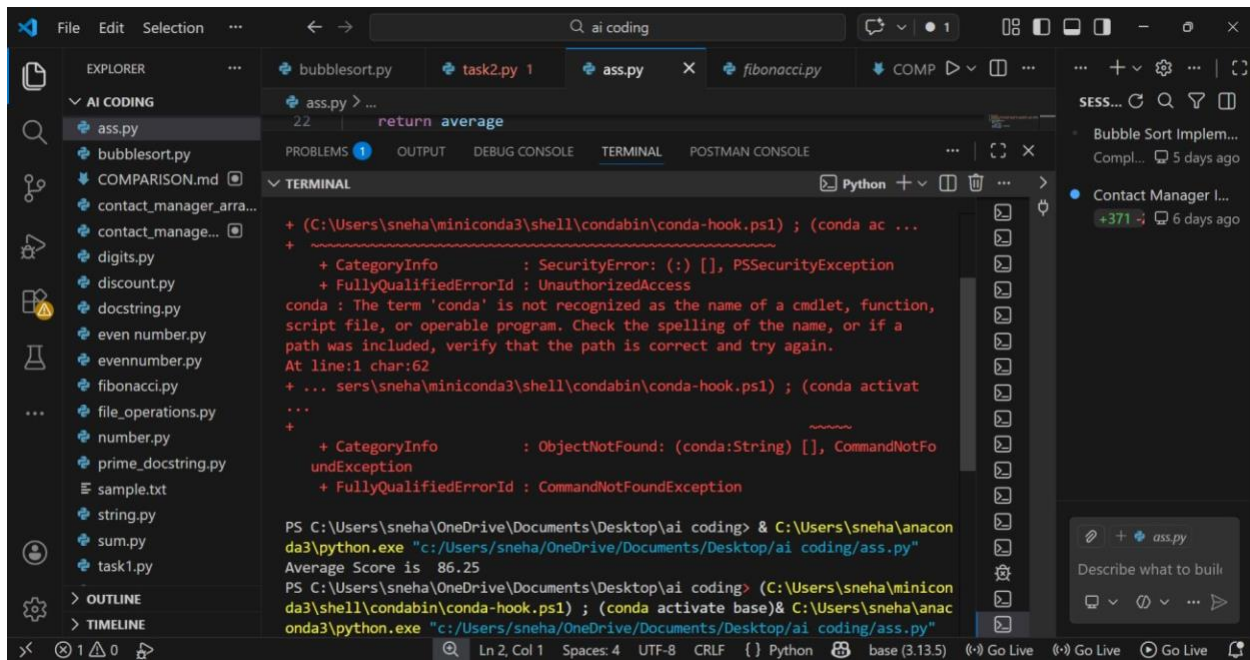
```
avrage marks = [85, 90, 78, 92] print("Average
```

```
Score is ", calc_average(marks)
```

Code:



Output:



Explanation:

-->Removed invalid characters — Bullet symbol (•) was causing a syntax error because Python accepts only valid code characters.

-->Fixed typo — Changed avrage to average so the correct variable is returned.

-->Corrected indentation — Proper indentation was added inside the function and loop (Python requires indentation).

-->Closed parenthesis — Added the missing) in the print() statement to avoid syntax error.

-->Ensured correct logic — Total marks are summed and divided by number of subjects to correctly compute the average.

Task 2: PEP 8 Compliance .Use AI to refactor Python code to follow PEP 8 style Guidelines.

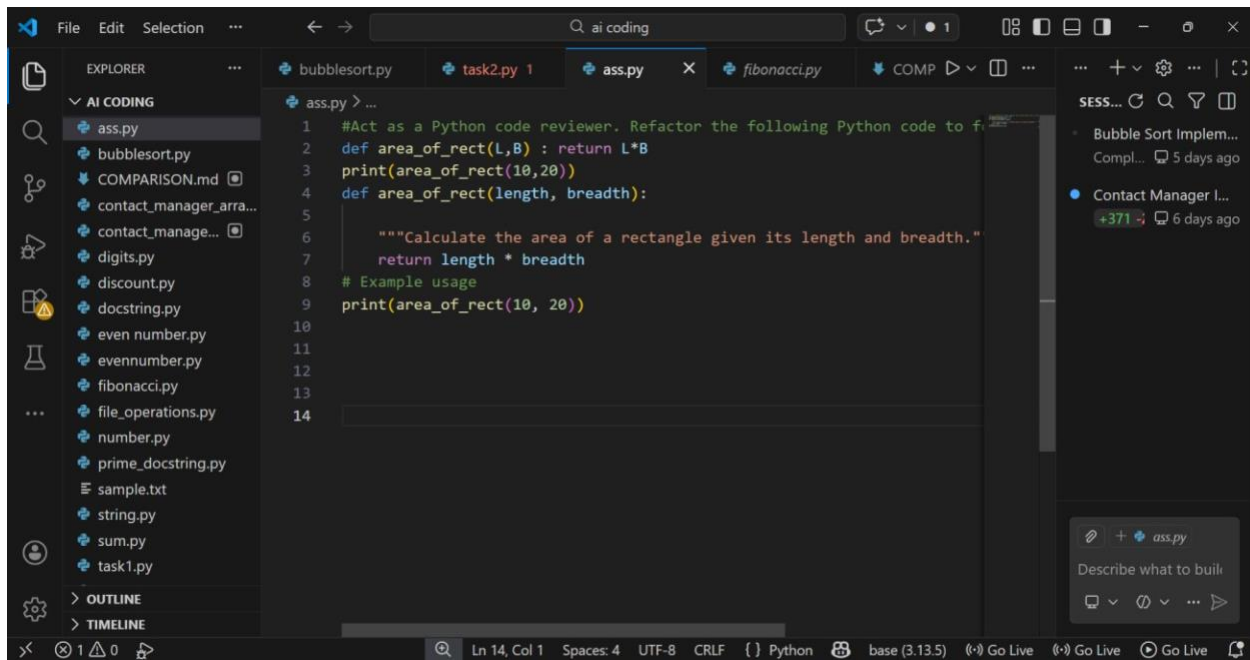
Prompt

Act as a Python code reviewer. Refactor the following Python code to follow PEP 8 style guidelines.

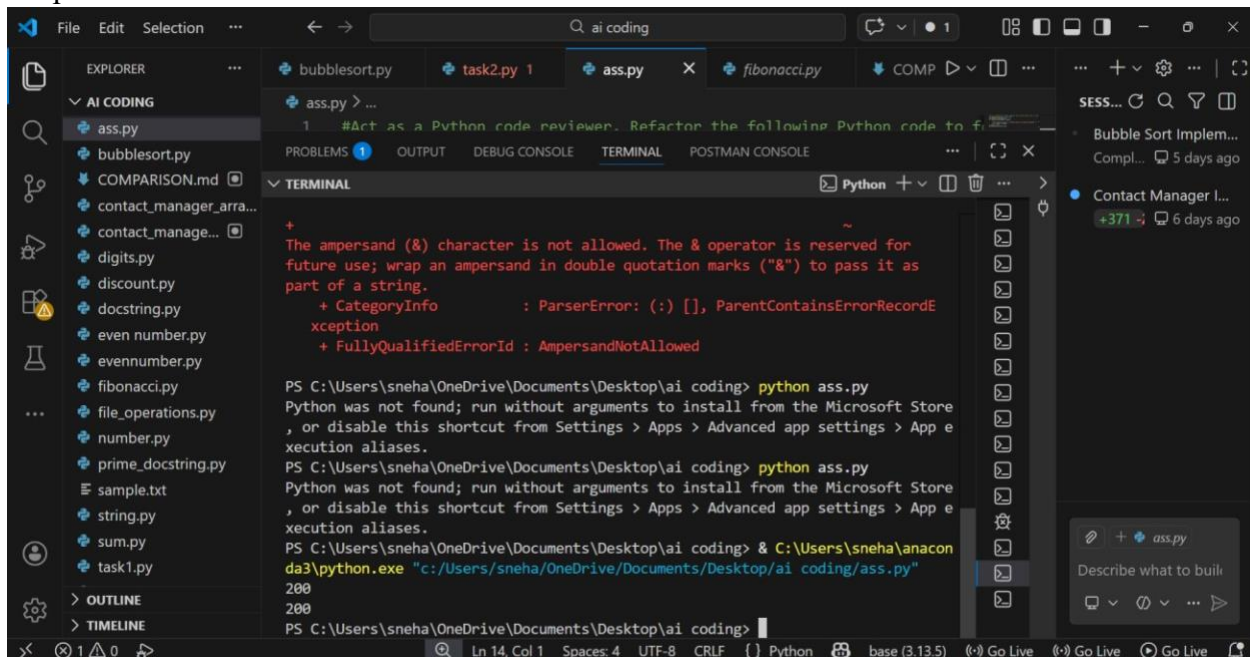
Code:

```
def area_of_rect(L,B) : return L*B
```

print(area_of_rect(10,20)) Code:



Output:



Explanation:

- >Used proper function naming — Changed variable names to lowercase with underscores (e.g., length, breadth) for readability.
- >Added correct spacing — Included spaces after commas and around operators (*) as per PEP 8 rules.

-->Split into multiple lines — Avoided writing the function in one line to improve clarity.
-->Removed extra spaces before colon — Ensured correct syntax formatting (def func():).
-->Improved readability — Structured the code neatly so it is easier to understand and maintain.

Task 3: Readability Enhancement. Use AI to make code more readable without changing its Logic.

Prompt:

You are given a Python program that works correctly but is poorly readable due to unclear variable names, lack of comments, and improper formatting.

Improve the readability of the code without changing its logic or output.

Replace unclear variable and function names with descriptive names.

Format the code clearly using proper indentation and spacing according to PEP 8 standards.

Do not modify the logic, calculations, or output of the program.

Input Code: def

c(x,y):

return x*y/100

a=200 b=15

print(c(a,b))

Output:

Readable, well-formatted Python code

Same functionality and output as the original program.

Code:

```
← → ai coding 1
bubblesort.py task2.py 1 ass.py fibonacci.py COMP ▾ □ ...
ass.py > ...
1 #You are given a Python program that works correctly but is poorly read.
2 '''lack of comments, and improper formatting.
3 Improve the readability of the code without changing its logic or output.
4 Replace unclear variable and function names with descriptive names.
5 Format the code clearly using proper indentation and spacing according
6 Do not modify the logic, calculations, or output of the program.
7 Input Code:
8 def c(x,y):
9     return x*y/100
10 a=200
11 b=15
12 print(c(a,b))
13 Output:
14 Readable, well-formatted Python code
15 Same functionality and output as the original program'''
16 def calculate_percentage(value, percentage):
17     """Calculate the percentage of a given value."""
18     return value * percentage / 100
19 # Sample values
20 total_amount = 200
21 percentage_value = 15
22 # Calculate and print the result
23 result = calculate_percentage(total_amount, percentage_value)
24 print(result)
```

Output:

```
ed\libs\debugpy\launcher' '53325' '--' 'c:\Users\sneha\OneDrive\Documents\Desktop\ai coding\ass.py'
30.0
PS C:\Users\sneha\OneDrive\Documents\Desktop\ai coding> █
```

Explanation:

-->Improved indentation — Proper spacing was added so Python can correctly understand the function block.

-->Used meaningful names — Replaced short names like c, x, y with descriptive names to make the code easier to understand.

-->Added comments — Inline comments explain what each part of the code does.

-->Formatted clearly — Code was arranged neatly with proper line breaks and spacing. Task

4: Refactoring for Maintainability. Use AI to break repetitive or long code into reusable

Functions.

Prompt:

Act as a Python refactoring assistant. Refactor the following code to improve maintainability by removing repetition and creating reusable functions.

- Identify repetitive code
- Break the code into reusable functions
- Keep the same functionality
- Improve readability and structure
- Provide the refactored modular Python code

```
students = ["Alice", "Bob", "Charlie"]
```

```
print("Welcome", students[0]) print("Welcome",
```

```
students[1]) print("Welcome", students[2])
```

Code:

```
ass.py  X  ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py  fibon  ▾  □  ...
ass.py > ...
1  #Act as a Python refactoring assistant. Refactor the following code to
2  '''• Identify repetitive code
3  • Break the code into reusable functions
4  • Keep the same functionality
5  • Improve readability and structure
6  • Provide the refactored modular Python code
7
8  Code:
9
10 students = ["Alice", "Bob", "Charlie"]
11 print("Welcome", students[0])
12 print("Welcome", students[1])
13 print("Welcome", students[2])'''
14 def welcome_students(students):
15     """Welcome each student in the list."""
16     for student in students:
17         print("Welcome", student)
18 # Sample list of students
19 students = ["Alice", "Bob", "Charlie"]
20 # Welcome the students
21 welcome_students(students)
22
```

Output:

```
op\ai coding\ass.py'
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\sneha\OneDrive\Documents\Desktop\ai coding>
Ln 24, Col 9  Spaces: 4  UTF-8  CRLF  { } Python  base (3.13.5)
```

Explanation:

-->Removed repetition — Instead of writing multiple print statements, repetitive code is replaced with a reusable approach.

-->Created a function — A function is introduced to handle welcoming students, making the code modular.

-->Used iteration — A loop is used to process all students, reducing code length.

-->Improved maintainability — Future changes (like modifying the message) can be done in one place.

-->Enhanced readability — The structure is cleaner and easier to understand.

Task 5: Performance Optimization Task. Use AI to make the code run faster. Sample Input Code:

Prompt:

Act as a Python performance optimization expert. Improve the following code to run faster while keeping the same output.

- Identify performance bottlenecks
- Use efficient techniques like list comprehensions or built-in functions
- Keep the logic unchanged
- Provide the optimized Python code
- Briefly explain the improvements

Code:

```
Find squares of numbers
nums = [i
for i in range(1,1000000)]
squares =
[]
for n in nums:
squares.append(n**2)
print(len(squares))
```

Code:

→

py ×  ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py  fibon ▶ ▾

s.py > ...

```
#Act as a Python performance optimization expert. Improve the following
'''• Identify performance bottlenecks
• Use efficient techniques like list comprehensions or built-in functions
• Keep the logic unchanged
• Provide the optimized Python code
• Briefly explain the improvements

Code:

# Find squares of numbers

nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)

print(len(squares))'''
# Optimized code using list comprehension for better performance
nums = [i for i in range(1, 1000000)]
squares = [n**2 for n in nums] # Using list comprehension to create the list
print(len(squares))
# Improvements:
```

← →

ai coding

🗨️ ▾ ● 1

ss.py ×

📄 ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py

📄 fibon ▾

ass.py > ...

```
# 1. Replaced the for loop with a list comprehension, which is generally
# 2. This approach reduces the overhead of multiple append calls and al

"""
This implementation uses a singly linked list to store contacts.
Efficient deletion but requires sequential access for searching.
"""

class Contact:
    """Represents a single contact with name and phone number."""

    def __init__(self, name, phone):
        self.name = name
        self.phone = phone

    def __repr__(self):
        return f"Contact(name='{self.name}', phone='{self.phone}')"

    def __str__(self):
        return f"Name: {self.name}, Phone: {self.phone}"

class Node:
```

🔍

Ln 166, Col 17

Spaces: 4

UTF-8

CRLF

{ }

Python

🐍

base (3.13.5)

⌕

```
← → ai coding
ass.py X ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py
fibon ▾

ass.py > ...
5
6 class Node:
7     """Node in the linked list containing a contact."""
8
9     def __init__(self, contact):
10         self.contact = contact
11         self.next = None
12
13 class LinkedListContactManager:
14
15     """Contact manager using a singly linked list for storage."""
16
17     def __init__(self):
18         """Initialize an empty linked list."""
19         self.head = None
20         self.count = 0
21
22     def add_contact(self, name, phone):
23         """
24         Add a new contact to the manager.
25
26         Time Complexity: O(n) - must check for duplicates by traversing
27         Space Complexity: O(1) for new node
28
```

Ln 166, Col 17 Spaces: 4 UTF-8 CRLF { } Python base (3.13.5)

← → ai coding 1

ass.py X ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py fibon ▾

ass.py > ...

```
class LinkedListContactManager:
    def add_contact(self, name, phone):
        # Check for duplicate names
        if self.search_contact(name) is not None:
            print(f"Error: Contact '{name}' already exists.")
            return False

        contact = Contact(name, phone)
        new_node = Node(contact)

        # Add to the beginning of the list
        new_node.next = self.head
        self.head = new_node
        self.count += 1

        print(f"✓ Contact '{name}' added successfully.")
        return True

    def search_contact(self, name):
        """Search for a contact by name.

        Time Complexity: O(n) - linear search through the linked list
```

Ln 166, Col 17 Spaces: 4 UTF-8 CRLF { } Python base (3.13.5) Go

```
← → ai coding
ass.py × ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py fibon ▾
ass.py > ...
class LinkedListContactManager:
    def display_all_contacts(self):
        Display all contacts in the manager.
        Time Complexity: O(n) - must traverse the entire list
        """
        if self.head is None:
            print("No contacts available.")
            return

        print("\n--- All Contacts (Linked List) ---")
        current = self.head
        index = 1
        while current is not None:
            print(f"{index}. {current.contact}")
            current = current.next
            index += 1
        print()

    def get_contact_count(self):
        """Return the total number of contacts in the manager.

        Time Complexity: O(1) - count is maintained as a variable
        """
        return self.count

Ln 166, Col 17 Spaces: 4 UTF-8 CRLF { } Python base (3.13.5)
```

Output:

```
\OneDrive\Documents\Desktop\ai coding'; & 'C:\Users\sneha\anaconda3\python.exe'
'c:\Users\sneha\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundl
ed\libs\debugpy\launcher' '56548' '--' 'c:\Users\sneha\OneDrive\Documents\Deskt
op\ai coding\ass.py'
999999
PS C:\Users\sneha\OneDrive\Documents\Desktop\ai coding> ^C
PS C:\Users\sneha\OneDrive\Documents\Desktop\ai coding>
PS C:\Users\sneha\OneDrive\Documents\Desktop\ai coding> cd 'c:\Users\sneha
```

Explanation:

->Reduced unnecessary loops — Replaced the manual loop with a list comprehension which is faster in Python.

-->Improved execution speed — List comprehensions are optimized internally, reducing overhead.

-->Simplified code — Fewer lines make the code cleaner and easier to maintain.

-->Lower memory overhead — Direct computation avoids extra intermediate steps.

-->Maintained same output — The logic (calculating squares and counting them) remains unchanged.

Task 6: Complexity Reduction. Use AI to simplify overly complex logic.

Prompt:

Act as a Python code simplification expert. Simplify the following code by reducing unnecessary complexity while keeping the same logic and output.

- Replace deeply nested if-else statements with cleaner logic (e.g., elif or mapping)
- Improve readability and structure
- Keep functionality unchanged
- Provide the simplified Python code
- Briefly explain the improvements

Code:

```
def grade(score): if
```

```
score >= 90:
```

```
return "A"
```

```
else: if score
```

```
>= 80:
```

```
return "B"
```

```
else: if score
```

```
>= 70:
```

```
return "C"
```

```
else: if score  
    >= 60: return  
    "D" else:  
    return "F"  
Code:
```

← →

ai coding

🗨️ ▾ ● 1

🔍

ass.py ×

ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py

fibon ▶ ▾

🔍

ass.py > ...

```
#Act as a Python code simplification expert. Simplify the following code
'''• Replace deeply nested if-else statements with cleaner logic (e.g.,
• Improve readability and structure
• Keep functionality unchanged
• Provide the simplified Python code
• Briefly explain the improvements

Code:

def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"'''
def grade(score):
```

🔍

Ln 39, Col 30 Spaces: 4 UTF-8 CRLF { } Python 🤖 base (3.13.5) (⌂) Go Li

```
← → ai coding
ass.py X ass.py (File Saved • February 25, 2026 at 9:43 AM) ↔ ass.py fibon ▶ ▢
ass.py > ...
4 def grade(score):
5     if score >= 90:
6         return "A"
7     elif score >= 80:
8         return "B"
9     elif score >= 70:
10        return "C"
11    elif score >= 60:
12        return "D"
13    else:
14        return "F"
15    #example usage
16    print(grade(95)) # Output: A
17    print(grade(85)) # Output: B
18    print(grade(75)) # Output: C
19    print(grade(65)) # Output: D
20    # Improvements:
21    # 1. Replaced deeply nested if-else statements with elif for better readability
22    # 2. The logic remains unchanged, ensuring the same output for the same input
23
24
25
26
Ln 39, Col 30 Spaces: 4 UTF-8 CRLF { } Python base (3.13.5) Go
```

Output:

```
her' '56512' '--' 'c:\Users\sneha\OneDrive\Documents\Desktop\ai coding\ass.p
A
B
C
D
PS C:\Users\sneha\OneDrive\Documents\Desktop\ai coding> █
```

Explanation:

->Removed deep nesting — Replaced multiple nested if-else blocks with a simpler structure to make the code easier to follow.

- >Used elif statements — This reduces unnecessary checks once a condition is satisfied. -
- >Improved readability — The logic becomes clearer and easier to understand at a glance.
- >Simplified maintenance — Future changes to grading rules can be made more easily.
- >Kept functionality same — The grading output remains exactly the same for all scores.